2048 游戏智能体实验报告

> 课程：人工智能概论
> 题目：采用人工智能方法实现 2048 游戏决策与优化
> 负责人：[redacted]
> 组员1 ：[redacted]
> 组员2 ：[redacted]

摘要

本实验围绕 2048 游戏智能体展开，目标是依次完成启发式决策、搜索增强、进化计算调参、监督学习价值拟合，以及搜索增强监督学习-强化学习框架的探索。项目并非从零实现全部基础设施，而是在开源项目 `Quentin18/gymnasium-2048` 的 Gymnasium 环境基础上进行二次开发，主要新增了多类 agent、评估脚本、训练脚本和实验产物管理。

实验的主线可以概括为：先用人工特征构造一个可解释的一步启发式评价函数，再用 Expectimax 将“一步贪心”推广为“随机节点求期望、玩家节点取最大”的前向搜索；之后使用进化算法自动搜索评价函数权重；最后将搜索得到的 afterstate 价值作为监督信号，训练 CNN 近似价值函数，并尝试把这种“搜索改进后的价值标签”继续用于强化式自举训练。实验结果表明，expectimax 分支中早期 SL1 动作概率网络在直接贪心评估下效果有限：100 局均分 `9173.80`，2048 达成率仅 `1.0%`；而基于 afterstate value 的 SL2 网络在 `depth=1` 搜索下取得了本实验中最好的综合表现：100 局均分 `36659.76`，2048 达成率 `83.0%`，4096 达成率 `21.0%`，已经超过手工启发式 Expectimax depth1，并接近甚至略高于本地记录中的 Expectimax depth2 均分。相比之下，后续 RLSL 自举训练虽然思路上更接近 Bellman 最优更新，但在当前实现和训练预算下没有稳定超过监督学习初始化模型。

本项目的全部代码，包括历史开发与测试都一并同步在了github上：
https://github.com/Lrencheng/gym-2048

小组分工

- 程东联（负责人）负责进化算法、监督学习、强化学习框架搭建等主要算法工作、项目管理和文档编写
- 史玥（组员1）负责强化学习调试、项目环境配置、代码测试与调试工作
- 李建坤（组员2）负责简单启发式算法模块、代码接口对齐等工作

1. 任务目标与需求分析

1.1 课程任务要求

根据要求，本次实践任务要求完成以下内容：

1. 实现 2048 游戏系统。棋盘为 4×4，每次合法移动后随机生成数字块，其中 2 的生成概率为 90%，4 的生成概率为 10%。
2. 采用启发式搜索算法设计 2048 决策策略。策略需要模拟上下左右四个动作并用启发式评价函数选择动作。
3. 采用进化计算方法优化策略参数，使平均得分、最大合成方块和平均生存步数得到提升。
4. 采用监督学习训练人工神经网络进行 2048 决策，可使用启发式搜索策略采集标记数据。
5. 采用强化学习方法继续优化神经网络模型或决策策略，并说明状态、动作、奖励、训练过程和结果。

1.2 本实验的技术路线

本实验将 2048 决策问题视为一个带随机转移的序贯决策问题。设当前棋盘状态为 s，玩家动作 $a \in \{\text{up}, \text{right}, \text{down}, \text{left}\}$。执行动作后先得到一个不含随机新块的中间状态，即 afterstate：

$$x = T(s, a), \qquad r = R(s, a)$$

随后环境在空格中随机生成 2 或 4，得到下一状态 s'。因此一个完整步长可以写为：

$$s \xrightarrow{\text{动作}a} x \xrightarrow{\text{随机节点}} s'$$

2048 的随机性不来自玩家动作本身，而主要来自 afterstate 后的随机生成。这个结构非常适合使用 afterstate 价值函数：

$$U(x) = \mathbb{E}_{s' \sim P(\cdot|x)}[V(s')]$$

若使用 Bellman 最优思想，状态价值和 afterstate 价值之间可以写成：

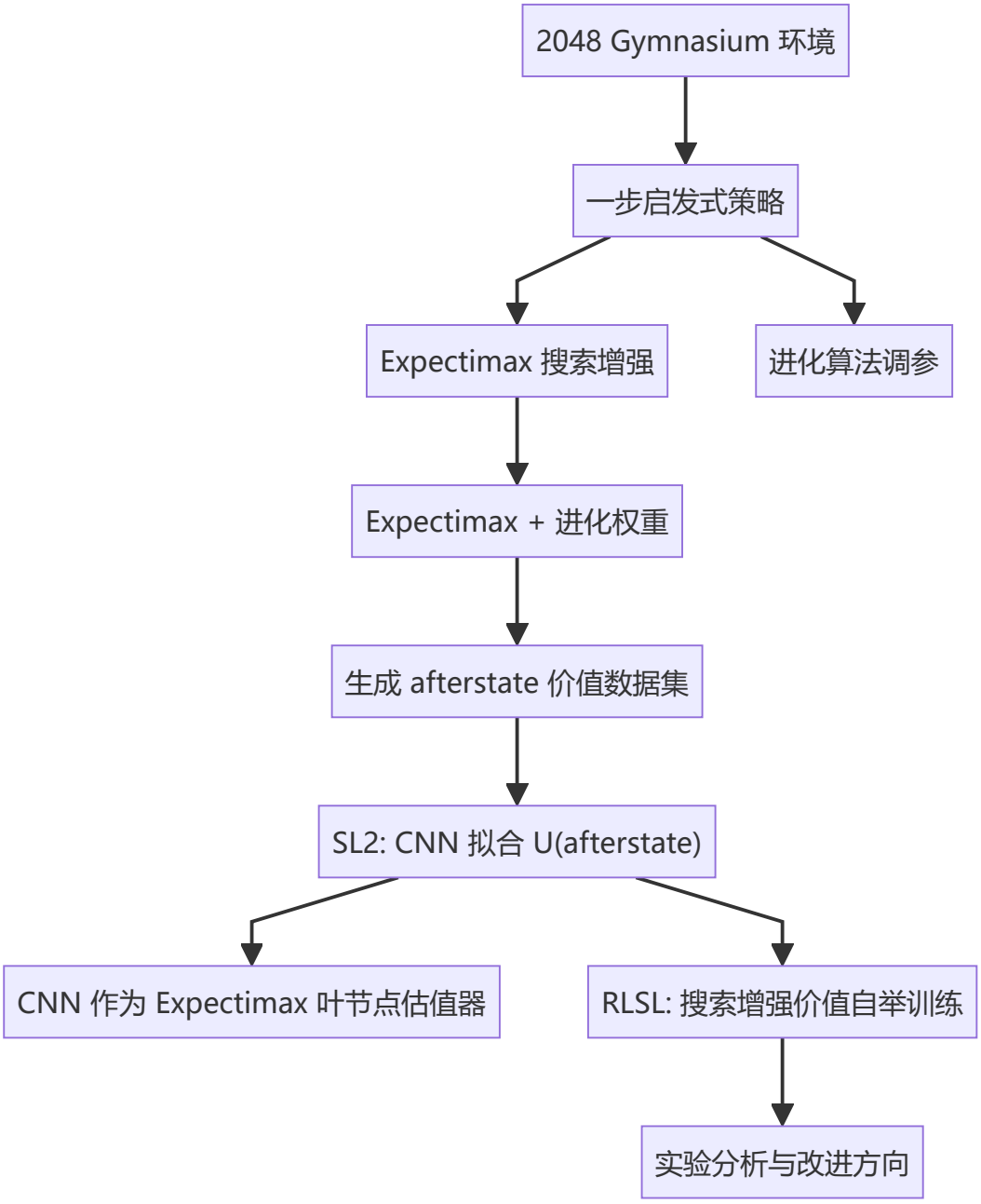
$$V^*(s) = \max_a \left[R(s, a) + U^*(T(s, a))\right]$$

$$U^*(x) = \mathbb{E}_{s' \sim P(\cdot|x)} \left[\max_{a'} \left(R(s', a') + U^*(T(s', a'))\right)\right]$$

这也是本实验从启发式、Expectimax 到 SL2/RLSL 的共同理论骨架：

人工启发式给 $U(x)$ 一个手工近似，Expectimax 用搜索增强这个近似，监督学习让神经网络拟合搜索后的 $U(x)$，RLSL 则试图用当前网络经过搜索得到更强标签，再反过来训练下一轮网络。

整体流程如下：



1.3 项目结构

本实验主要代码和结果分布如下：

模块	路径	作用
游戏环境	`gymnasium-2048/src/gymnasium_2048/envs/`	2048 Gymnasium 环境，含移动、合并和随机生成逻辑
一步启发式	`gymnasium-2048/src/gymnasium_2048/agents/heuristic/`	人工特征评价函数和一步贪心策略
Expectimax	`gymnasium-2048/src/gymnasium_2048/agents/expectimax/`	afterstate 递归搜索、随机节点展开、教师数据生成
进化算法	`gymnasium-2048/src/gymnasium_2048/agents/evolution/`	对 heuristic/expectimax 权重进行遗传算法优化
监督学习	`gymnasium-2048/src/gymnasium_2048/agents/supervised_cnn/`	SL2:CNN 价值网络回归和搜索集成策略

模块	路径	作用
RLSL	`gymnasium-2048/src/gymnasium_2048/agents/RLSL/`	search-improved afterstate value learning 框架
N-Tuple	`gymnasium-2048/src/gymnasium_2048/agents/ntuple/`	TD/Q-learning 的 N-Tuple 网络实现
实验图表	`report/assets/`	本报告引用的全部图片和汇总表

2. 实验总览

2.1 棋盘表示与评价指标

环境内部并不直接存储实际数字，而是存储指数值。若棋盘格 $b_{ij} = 0$ 表示空格，$b_{ij} = 1$ 表示实际方块 2，$b_{ij} = 2$ 表示实际方块 4，以此类推。实际数值为：

$$\text{tile}(b_{ij}) = \begin{cases} 0, & b_{ij} = 0 \\ 2^{b_{ij}}, & b_{ij} > 0 \end{cases}$$

实验主要使用以下评价指标：

指标	含义
`mean_score`	多局平均总得分，是最主要的性能指标
`best_score`	评估局中的最高分，用于观察偶然上界
`mean_steps`	平均存活步数，反映策略是否能长期维持棋盘可操作性
`max_tile`	评估过程中达到的最大方块
`2048_rate`	达到 2048 的比例
`4096_rate`	达到 4096 的比例
`illegal_action_rate`	非法动作比例，用于检查策略实现是否正确

2.2 主要实验结果总表

表中结果来自本地已有实验记录、模型评估 JSON、以及本报告补跑的轻量基准。不同实验的权重版本和搜索配置略有历史差异，严格横向比较时主要关注同组实验内的相对变化。其中 SL1 这一行是本次在 `expectimax` 分支补跑的直接贪心评估，未叠加 Expectimax 搜索。

方法	局数	`mean_score`	`best_score`	`mean_steps`	`max_tile`	`2048_rate`	`4096_rate`
一步启发式贪心	100	11778.44	33916	698.37	2048	0.150	0.000
Expectimax depth0 + 手工启发式	100	8351.36	30672	531.07	2048	0.040	0.000
Expectimax depth1 + 手工启发式	100	25271.72	70036	1330.32	4096	0.570	0.090
Expectimax depth2 + 手工启发式	100	34527.52	74380	1747.89	4096	0.790	0.180
SL1 动作概率网络贪心策略	100	9173.80	26084	592.04	2048	0.010	0.000

方法	局数	mean_score	best_score	mean_steps	max_tile	2048_rate	4096_rate
SL2 价值网络 + depth1	100	36659.76	73720	1838.55	4096	0.830	0.210
RLSL multi_eps 最好一轮 depth0 评估	50	23497.92	50928	1253.04	4096	0.580	0.020

可以看出，搜索深度从 depth1 到 depth2 带来了明显收益；而 SL2 通过学习 depth2 教师的 afterstate 价值，使 `depth=1` 的推理成本获得了接近 depth2 的效果。RLSL 自举训练目前没有稳定提升，后文会单独分析原因。

3. 简单启发式算法

3.1 思路分析

最早完成的是一步启发式贪心策略。它不向未来展开随机节点，也不做多层搜索，而是对当前状态的四个合法动作逐一模拟：

$$x_a = T(s, a)$$

然后用一个手工评价函数 $H(x_a, r_a)$ 直接给 afterstate 打分，并选择分数最大的动作：

$$\pi(s) = \arg\max_a H(T(s, a), R(s, a))$$

这种方法的优点是计算快、可解释、实现稳定；缺点是视野短，只能评价一步之后的局面，容易被即时合并奖励诱导，也不能显式处理随机生成新块后的风险。

本实验的启发式评价函数由多个特征加权组合：

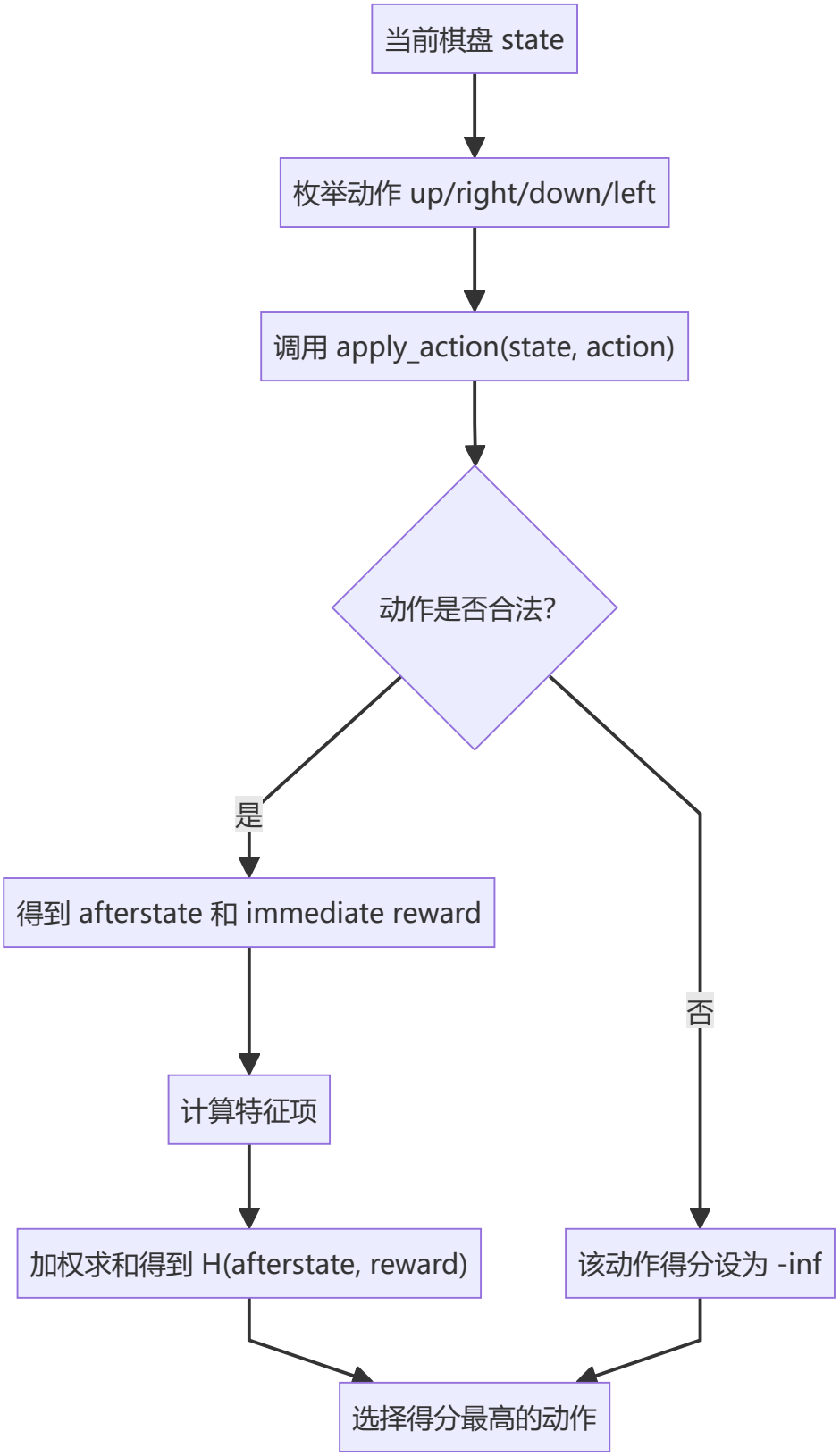
$$H(x, r) = w_e f_{\text{empty}}(x) + w_s f_{\text{smooth}}(x) + w_m f_{\text{mono}}(x) + w_c f_{\text{corner}}(x) + w_p f_{\text{merge}}(x) + w_b f_{\text{edge}}(x) + w_r g(r)$$

其中：

特征	直觉	作用
空格数 `empty_cells`	空格越多，后续可操作空间越大	提高存活性
平滑度 `smoothness`	相邻数字越接近，越容易合并	减少棋盘断层
单调性 `monotonicity`	大数沿行列有序排列更稳定	帮助形成合并链
角落最大数 `corner_max`	最大块在角落更不容易被破坏	保护核心大块
合并潜力 `merge_potential`	相邻相等块越多，近期收益越高	鼓励可合并结构
边缘奖励 `edge_bonus`	大数靠边更稳定	减少大块游离
即时奖励 `reward`	当前动作实际获得的分数	保留得分目标

3.2 算法架构

对应代码在 `gymnasium-2048/src/gymnasium_2048/agents/heuristic/`。策略入口是 `HeuristicPolicy.predict(state)`，特征函数在 `features.py` 中。



3.3 实验结果与分析

当前 `heuristic/configs/evaluate.yaml` 下补跑 100 局，结果如下：

指标	数值
平均分	11778.44
最高分	33916

指标	数值
平均步数	698.37
最大块	2048
2048 达成率	15.0%
4096 达成率	0.0%
非法动作率	0.0%

一步启发式可以稳定完成基本游戏决策，且不会产生非法动作，说明动作模拟和特征评价的工程实现是可靠的。但它的上限比较明显：平均步数不足 700，2048 达成率只有 15%。从策略行为上看，一步贪心只知道“当前合并是否好”“当前空格是否多”，不知道随机新块生成后会不会破坏局面，也不知道某个短期得分动作是否会在两三步后把最大块移出角落。因此，它适合作为后续搜索和学习算法的叶节点估值器，而不是最终最优策略。

4. Expectimax + 简单启发式算法

4.1 思路分析

2048 中玩家动作是可控的，但每步之后新方块的位置和数值是随机的。因此，简单 minimax 不适合直接使用，因为环境不是对抗性对手，而是概率随机过程。Expectimax 的核心思想是：

1. 在玩家节点选择价值最大的动作；
2. 在随机节点按照概率对后续状态价值求期望；
3. 在搜索深度到达上限后，用启发式函数或神经网络估计叶节点价值。

在本实验的 afterstate 表示下，Expectimax 可以写成：

$$U_0(x) = h(x)$$

$$U_d(x) = \sum_{s'} P(s' \mid x) \max_{a'} \left[R(s', a') + U_{d-1}(T(s', a')) \right], \quad d \geq 1$$

根状态动作评分为：

$$Q_d(s, a) = R(s, a) + U_d(T(s, a))$$

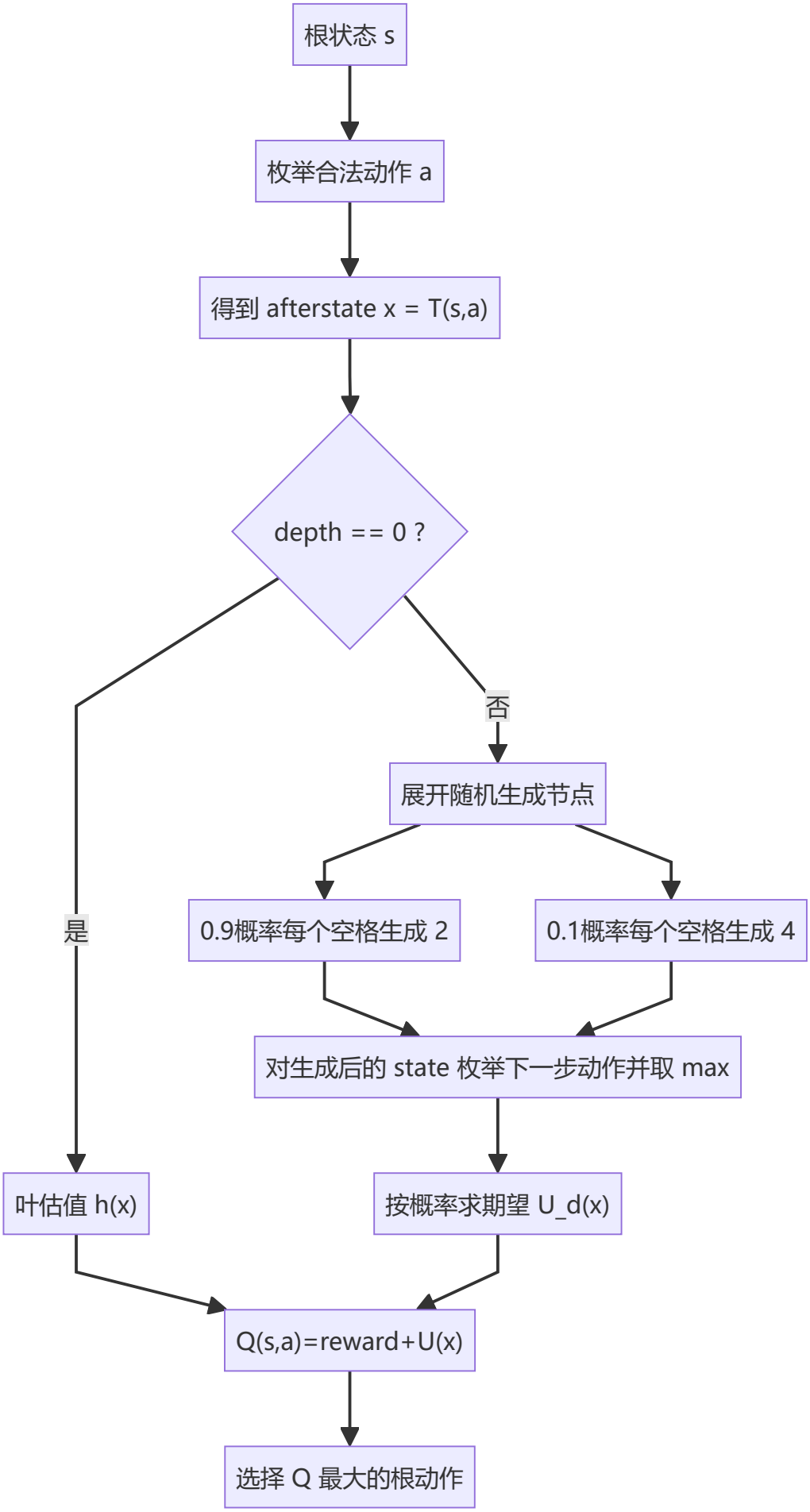
策略选择：

$$\pi_d(s) = \arg\max_a Q_d(s, a)$$

这个递推与 Bellman 最优方程的形式一致：随机生成部分对应期望，玩家决策部分对应最大化。与一步启发式相比，Expectimax 不只是“看一步之后的棋盘好不好”，而是进一步考虑“如果此时随机生成一个 2 或 4，下一轮还是否有好动作”。

4.2 算法架构

对应代码主要在 `gymnasium-2048/src/gymnasium_2048/agents/expectimax/search.py` 和 `policy.py`。


根状态 s
枚举合法动作 a
得到 afterstate x = T(s,a)
depth == 0 ?
是
否
展开随机生成节点
0.9概率每个空格生成 2
0.1概率每个空格生成 4
对生成后的 state 枚举下一步动作并取 max
叶估值 h(x)
按概率求期望 U_d(x)
Q(s,a)=reward+U(x)
选择 Q 最大的根动作

Trick: 在标准的2048-expectimax算法中，单层随机节点最多有 $2E$ 个分支，其中 E 为空格数；玩家节点最多有 4 个动作，随着深度的增加，单步step的计算量将指数级增长。为了控制复杂度，代码中支持 `chance_samples`。因此当空格较多时，可以只抽样一部分空格展开随机节点；当空格数低于阈值时，则完整枚举随机生成位置。

4.3 实验结果与分析

控制深度超参数，其他配置不变，三种搜索深度的代表性评估结果如下。

方法	mean_score	best_score	mean_steps	max_tile	2048_rate	4096_rate
Expectimax depth0 + 手工启发式	8351.36	30672	531.07	2048	4.0%	0.0%
Expectimax depth1 + 手工启发式	25271.72	70036	1330.32	4096	57.0%	9.0%
Expectimax depth2 + 手工启发式	34527.52	74380	1747.89	4096	79.0%	18.0%

对应评估分布图如下：

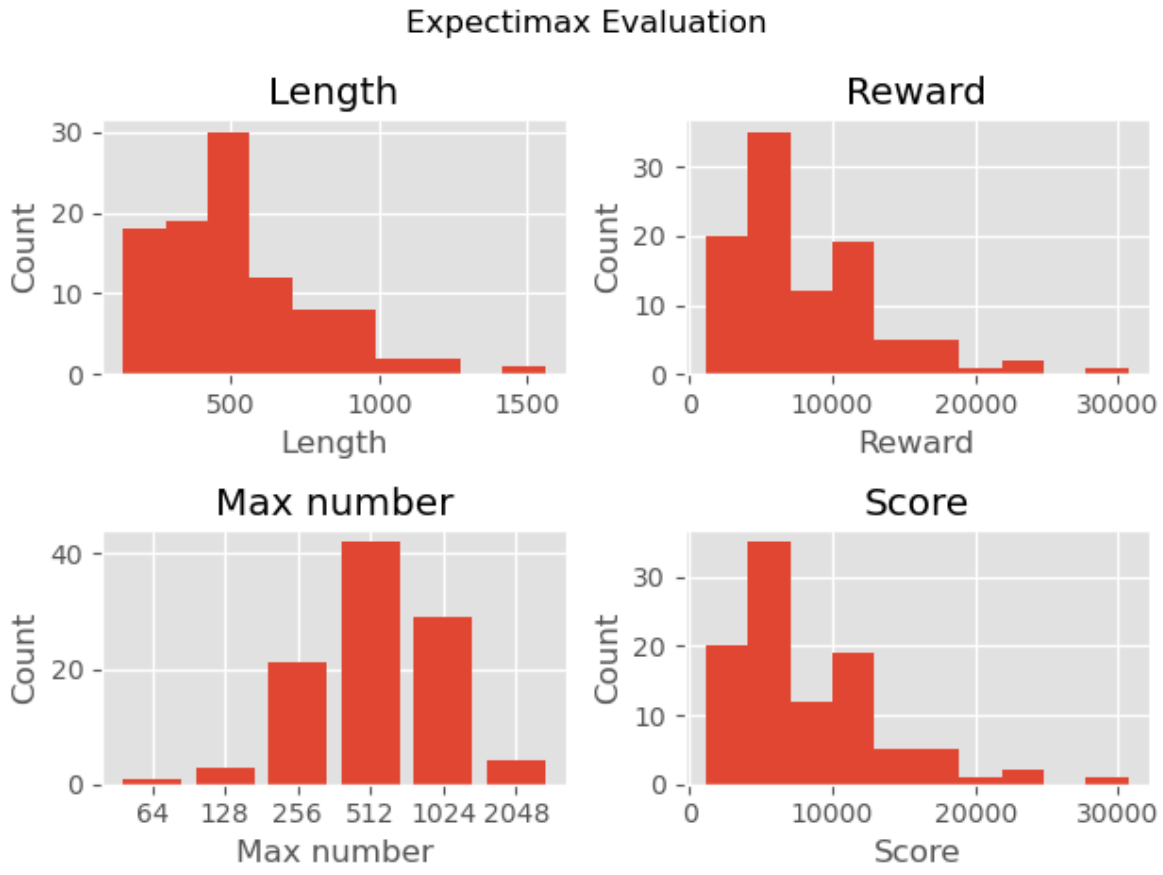


图4-1 Expectimax depth0 + 手工启发式的评估分布

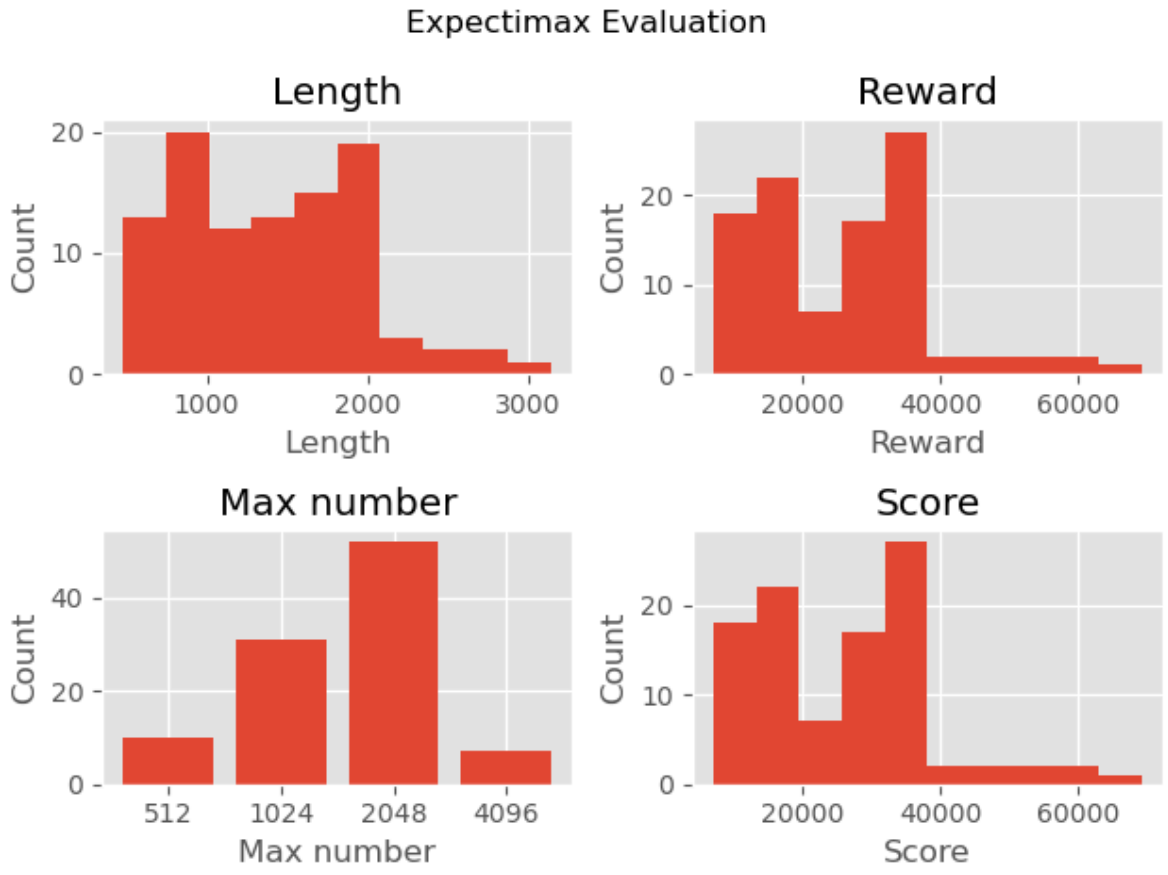


图4-2 Expectimax depth1 + 手工启发式的评估分布

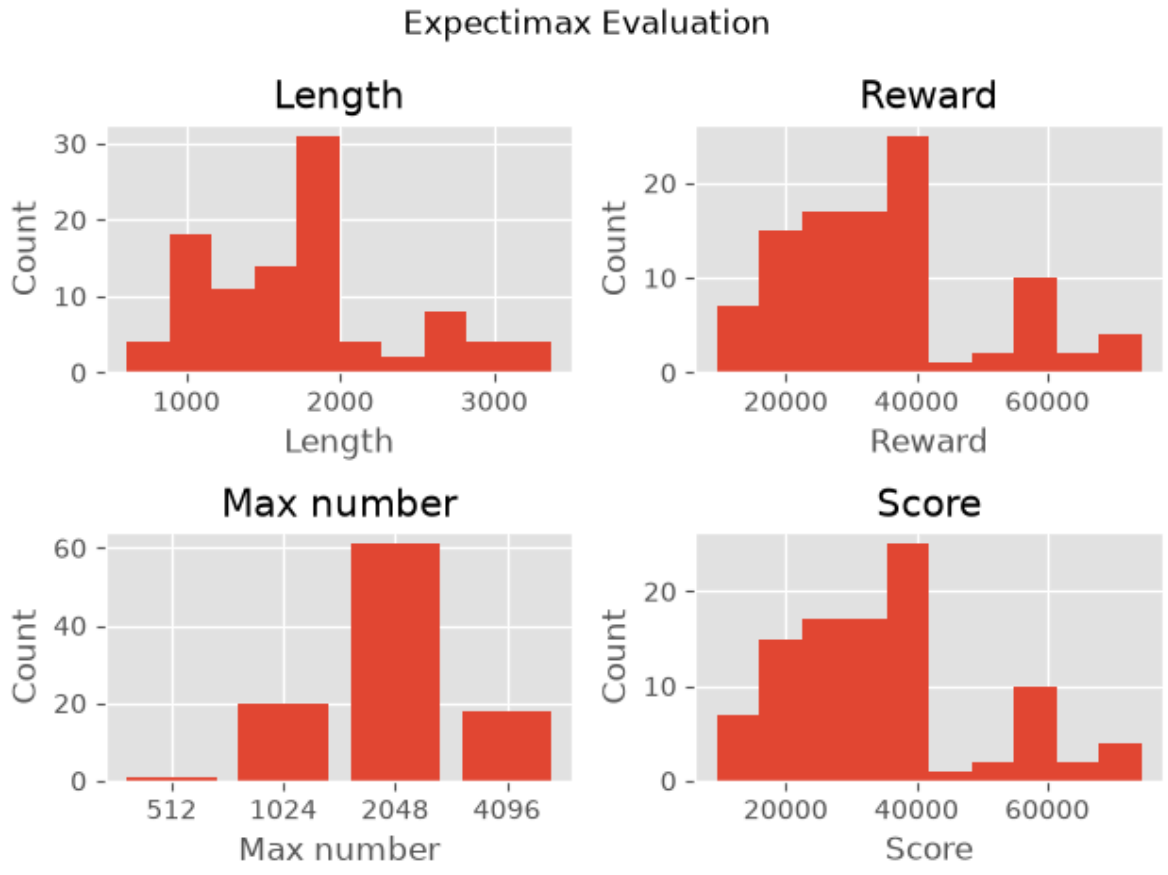


图4-3 Expectimax depth2 + 手工启发式的评估分布

可以看到，depth1 相比 depth0 有明显提升，平均分从 8351.36 提升到 25271.72，2048 达成率从 4.0% 提升到 57.0%。这说明"考虑一次随机生成和下一步最优动作"已经足以过滤掉大量短视动作。depth2 继续提升到 34527.52，2048 达成率达到 79.0%，4096 达成率达到 18.0%，说明多看一层可以显著改善高分局稳定性。

不过 Expectimax 的成本也很高。实验记录中 depth2 的 100 局运行时间约为 2044.68 秒，而 depth1 约为 234.16 秒。换句话说，depth2 的效果更好，但计算开销也高出一个数量级左右。这为后续监督学习提供了动机：

能否用神经网络学习 depth2 的叶节点估值能力，使较浅搜索也有接近深搜索的效果？

5. 进化算法更新权重

5.1 思路分析

启发式评价函数的结构是人工设计的，但各特征权重很难手工确定。比如空格数权重过高会导致策略过度保守，即使有高价值合并也不愿执行；即时奖励权重过高则会导致短视，优先合并当前能得分的块，而忽略最大块位置和棋盘结构。

因此，本实验用进化算法搜索权重向量：

$$\theta = (w_e, w_s, w_m, w_c, w_p, w_b, w_r)$$

对于 Expectimax 叶估值器，还扩展出：

$$\theta = (w_e, w_s, w_m, w_p, w_c, w_{\max}, w_{\text{snake}}, w_r)$$

每个候选权重在固定随机种子下运行若干局，使用平均分作为适应度：

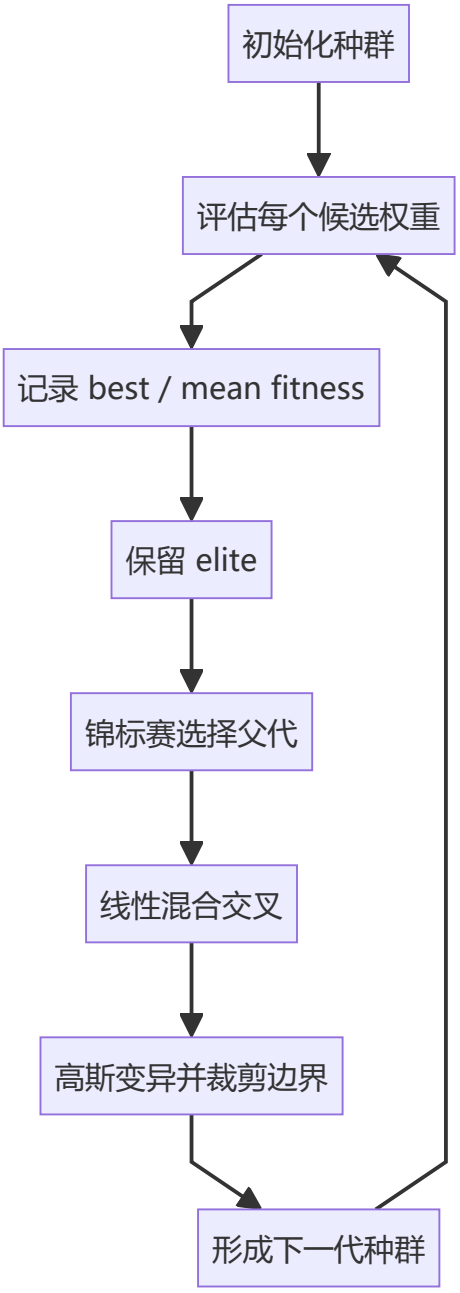
$$F(\theta) = \frac{1}{N} \sum_{i=1}^{N} \text{score}_i(\theta)$$

使用固定 episode seeds 可以降低候选之间因为环境随机性造成的比较噪声。

5.2 算法架构

进化算法实现位于 `gymnasium-2048/src/gymnasium_2048/agents/evolution/`。核心组件包括：

步骤	实现方式
初始化	在每个参数边界内均匀随机生成种群
适应度评估	每个候选权重运行若干局 2048，取平均分
选择	锦标赛选择 `tournament_size=3`
精英保留	每代保留适应度最高的若干候选
交叉	对两个父代做逐维线性混合：$c_i = \alpha_i p_i^{(1)} + (1-\alpha_i) p_i^{(2)}$
变异	以 `mutation_rate` 对参数加入高斯扰动
裁剪	将变异后参数裁剪回合法搜索区间



本实验进行了两类进化：

1. 直接优化一步启发式权重；
2. 优化 Expectimax 使用的叶节点启发式权重，并采用分阶段训练：先用低成本 depth1/chance sample 较少的设置粗调，再用 depth2 和更多随机节点样本复评。

5.3 实验结果与分析

一步启发式的大规模进化结果保存在 `gymnasium-2048/models/evolution_large/`。权重轨迹如下：

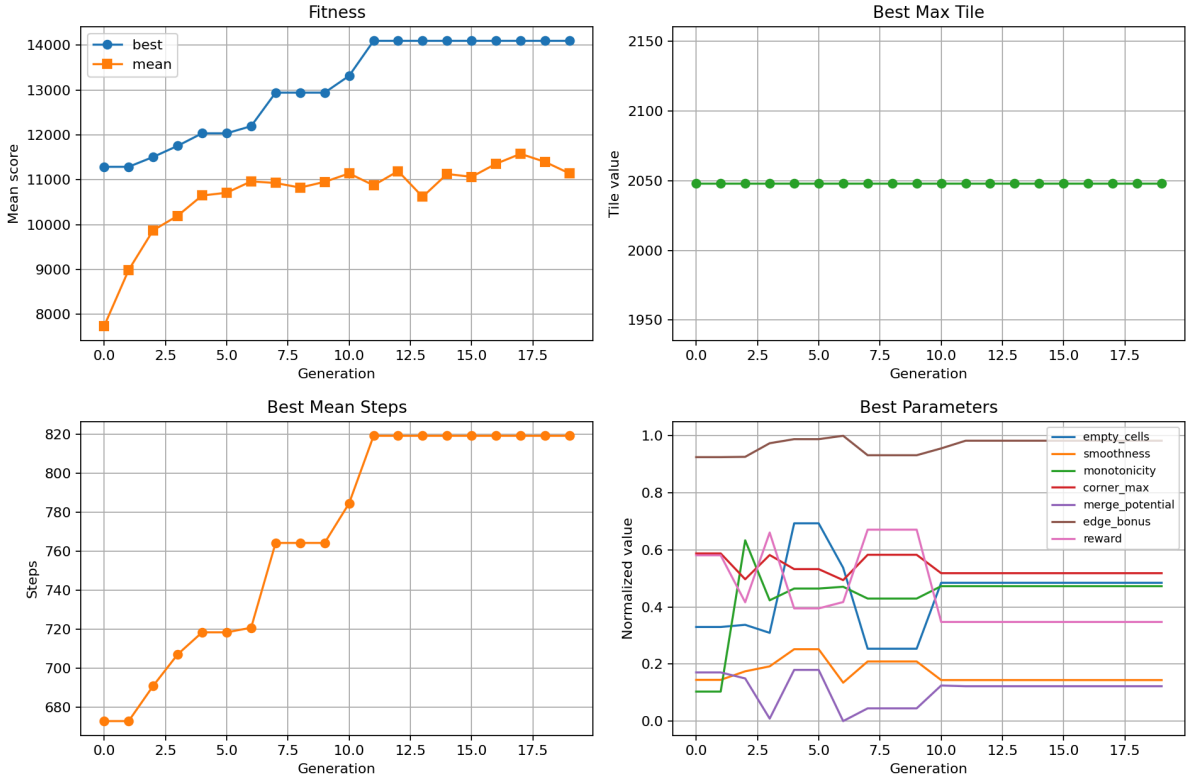


图5-1 一步启发式评价函数的进化权重轨迹

该训练从第 0 代最佳均分 11287.84 提升到第 19 代 14098.00，平均步数从 672.92 提升到 819.22。虽然最大块仍主要停留在 2048，但进化确实提高了同类一步策略的稳定性。

进化后的代表性权重如下：

参数	权重
`empty_cells`	12.271004
`smoothness`	0.859501
`monotonicity`	2.891370
`corner_max`	3.555273
`merge_potential`	2.732829
`edge_bonus`	0.098447
`reward`	1.021244

这些权重说明，进化算法最终仍然把“空格数”和“结构单调性”放在比较重要的位置，同时没有把即时奖励权重推得过高。这与 2048 的经验一致：高分局首先需要活得久，其次才是局部合并收益。

Expectimax 权重采用分阶段进化。Stage1 使用 depth1、较少 chance sample，主要快速筛出较优区域；Stage2 使用 depth2 和更多样本，在更接近最终评估条件的设置下细调。

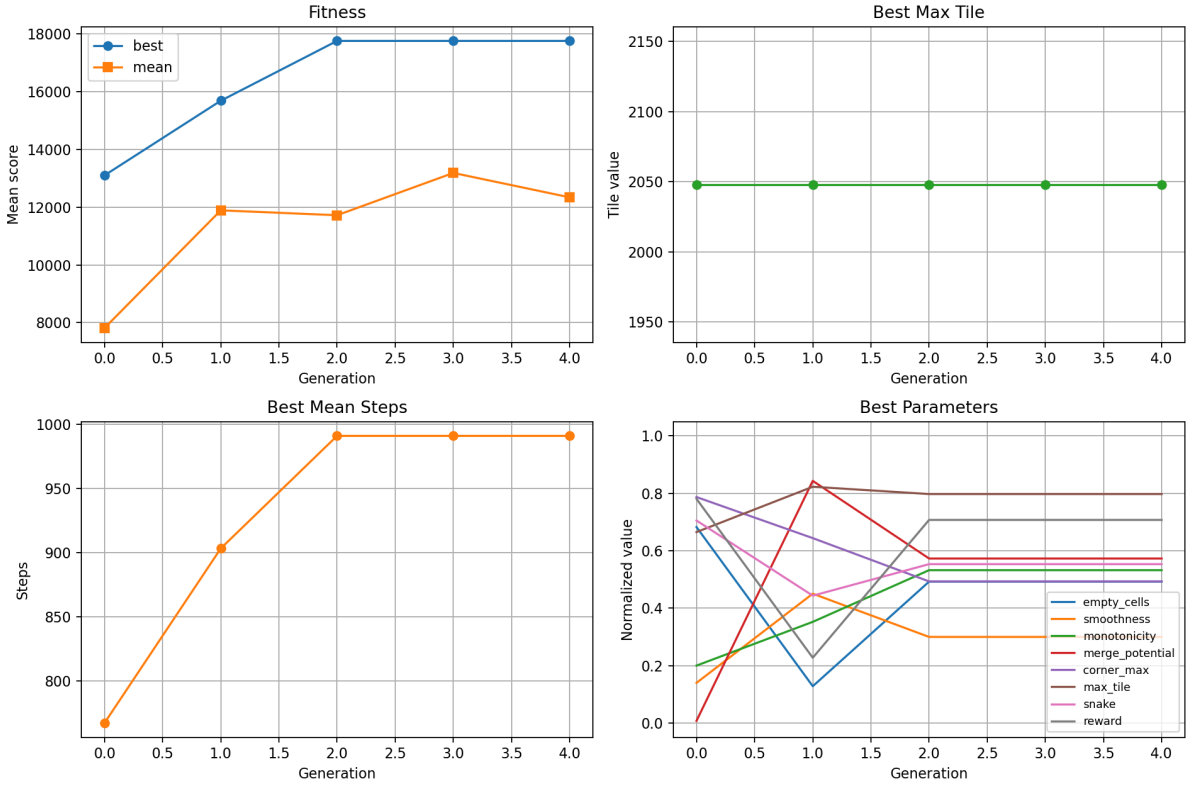


图5-2 Expectimax 权重分阶段进化的 Stage1 轨迹

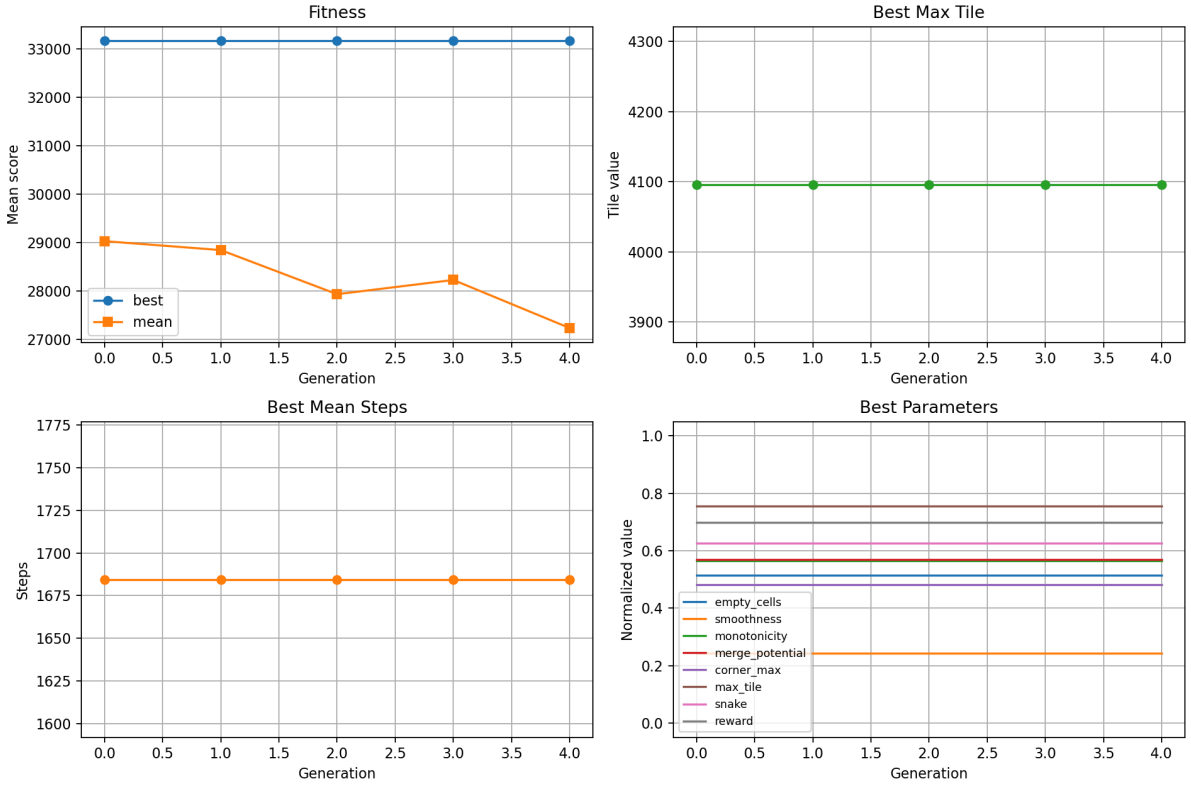


图5-3 Expectimax 权重分阶段进化的 Stage2 轨迹

Stage1 最佳均分达到 17760.00，Stage2 在 15 局/候选的设置下达到 33165.60，并在最终 50 局复评中得到最佳均分 29024.64。进化后的 Expectimax 权重强调 `monotonicity`、`merge_potential`、`snake` 和 `reward`，这说明在搜索已经帮助处理随机性的情况下，叶节点估值器更需要表达“局面形状”和“未来合并链”的质量。

不过，进化算法也有局限：每个候选需要实际运行多局游戏，评估成本很高；而 2048 随机性强，少量局数下的适应度噪声会很大。因此，进化适合做中等规模的权重粗调，但不适合作为无限制提高性能的唯一手段。

6. Expectimax + 监督学习

6.1 思路分析：从 SL1 到 SL2

监督学习部分经历了两条路线。

早期 SL1（位于 `expectimax` 分支）使用基于策略的思想，试图让神经网络直接学习教师策略的动作概率，也就是输入当前状态 s，输出四个动作的概率分布：

$$\pi_\theta(a \mid s) \approx \pi_{\text{teacher}}(a \mid s)$$

这种做法直观，但效果不理想。原因主要有三点：

1. 偏分布：2048 中很多状态只出现一次，状态分布很偏。
2. 理论上限：模仿学习一旦遇到训练集中没有覆盖的新局面，很容易失真。
3. 不确定性：当前状态 s 之后还包含随机生成新块的不确定性，不确定性对于很多基于策略的方法是致命的，这将使直接从 s 到动作概率的映射更难学。

因此，后续 SL2 使用基于价值的思想，改为学习 afterstate value，即输入玩家动作之后、随机生成之前的棋盘 x，输出该 afterstate 的未来价值：

$$U_\theta(x) \approx U_{\text{teacher}}(x)$$

这一转向更贴近 Bellman 最优方程。因为动作带来的确定性变化已经被折叠进 afterstate，网络只需要评估"这个中间局面经过随机生成后，未来还能拿多少分"。根状态选择动作时再使用：

$$a^* = \arg\max_a \left[R(s,a) + U_\theta(T(s,a)) \right]$$

6.2 算法架构

SL2 的数据集来自 `gymnasium-2048/data/expectimax_afterstates_400.npz`，由 `expectimax depth2 + 手工启发式 + 进化调参权重` 生成。数据集信息如下：

字段	数值
教师 episode 数	400
root state 数	610243
afterstate 样本数	2190477
搜索深度	depth2
reward transform	raw
target 均值	132.98
target 标准差	113.64
target 范围	-2.77 到 6295.15

数据生成时，对每个根状态枚举所有合法动作，保存：

$$(x_i, y_i) = (\text{afterstate}, U_{\text{expectimax-depth2}}(x_i))$$

网络结构为一个不使用池化的 CNN：

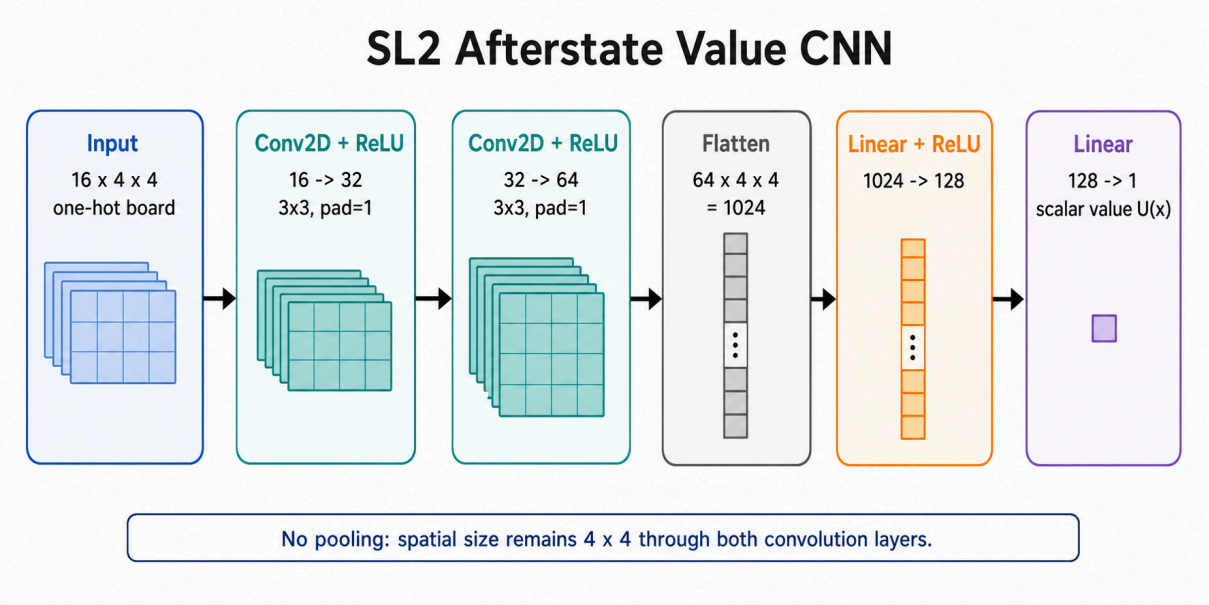
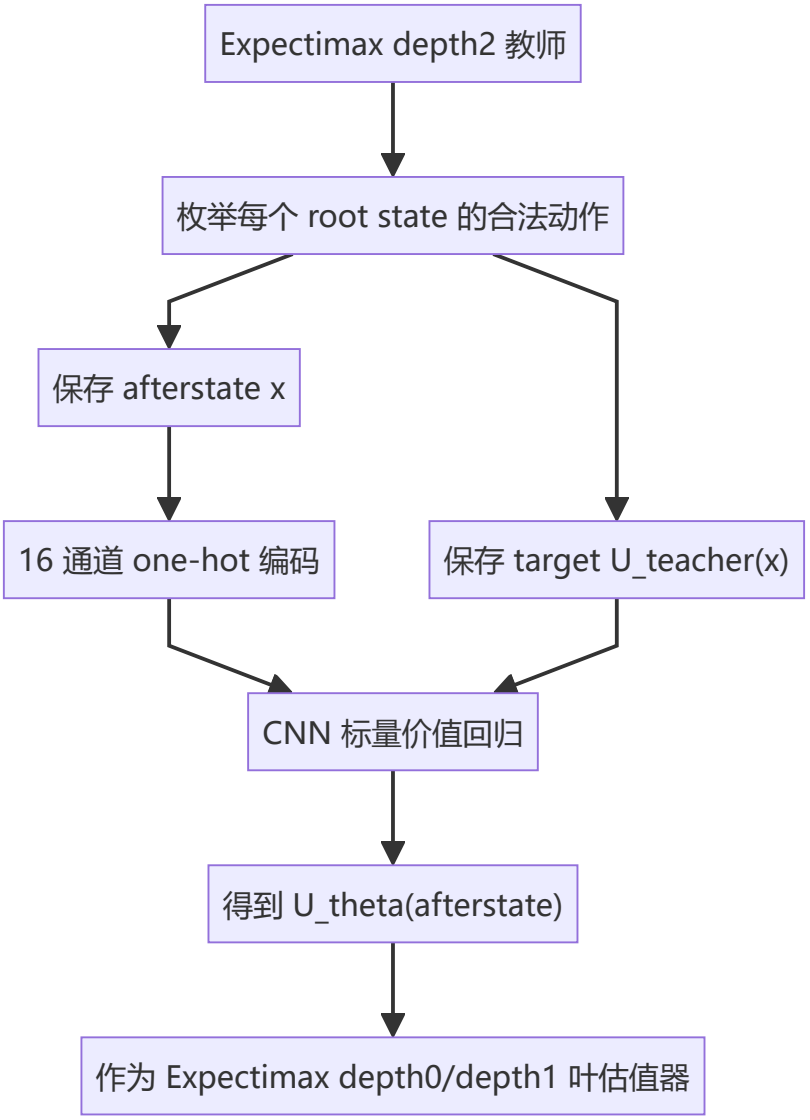


图6-1 SL2 afterstate value CNN 的网络结构示意图

训练目标为归一化后的 Huber loss：

$$\mathcal{L}(\theta) = \mathrm{Huber}\left(\frac{U_\theta(x) - \mu}{\sigma}, \frac{y - \mu}{\sigma}\right)$$

其中 μ, σ 是训练集 target 的均值和标准差。训练中使用了 **D4 棋盘对称增强**，即旋转和翻转棋盘，以减少数据分布偏差。



6.3 训练结果与分析

SL2 使用 `cnn_400eps_long` 配置训练 150 个 epoch，batch size 为 8192，学习率为 0.001，权重衰减为 0.0001，验证集比例为 0.1。训练曲线如下：

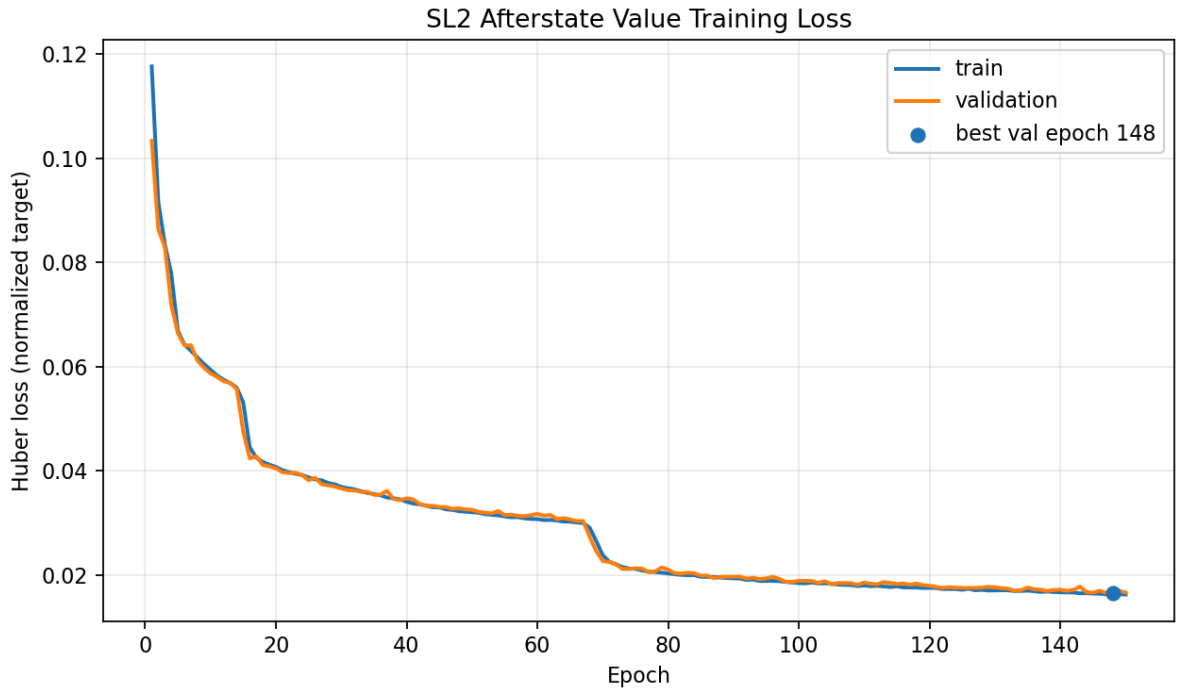


图6-2 SL2 afterstate value CNN 的训练与验证损失曲线

训练 loss 从第 1 轮的 0.1176 降到第 150 轮的 0.0163，验证 loss 最优出现在第 148 轮，为 0.0165。训练和验证曲线非常接近，说明在 D4 增强和目标归一化后，模型没有出现明显过拟合。第 15-16 轮、第 69-70 轮附近的 loss 阶跃下降，可能与优化器进入更好的参数区域有关。

6.4 评估结果与分析

SL1 与 SL2 的代表性结果如下。SL1 结果为本次回到 `expectimax` 分支补跑得到：checkpoint 为 `models/supervise/train4/checkpoints/best.pt`，评估 100 局，seed 为 42，使用 CPU。需要特别说明，虽然该分支的 `supervised_cnn/configs/evaluate.yaml` 中保留了 `depth` 字段，但 `evaluation.make_policy()` 只有在 `agent == "expectimax"` 时才构造 `ExpectimaxPolicy`；当 `agent == "supervised_cnn"` 时，它直接加载 `SupervisedCNNPolicy`。该策略对四个动作 logits 做合法动作 mask 后取 `argmax(masked_logits)`，所以这里报告的是 SL1 动作概率网络的直接贪心评估，不是 depth1 搜索。

方法	mean_score	best_score	mean_steps	max_tile	2048_rate	4096_rate
SL1 动作概率网络贪心策略	9173.80	26084	592.04	2048	1.0%	0.0%
一步启发式贪心	11778.44	33916	698.37	2048	15.0%	0.0%
SL2 afterstate value + depth1	36659.76	73720	1838.55	4096	83.0%	21.0%
Expectimax depth1 + 手工启发式	25271.72	70036	1330.32	4096	57.0%	9.0%
Expectimax depth2 + 手工启发式	34527.52	74380	1747.89	4096	79.0%	18.0%

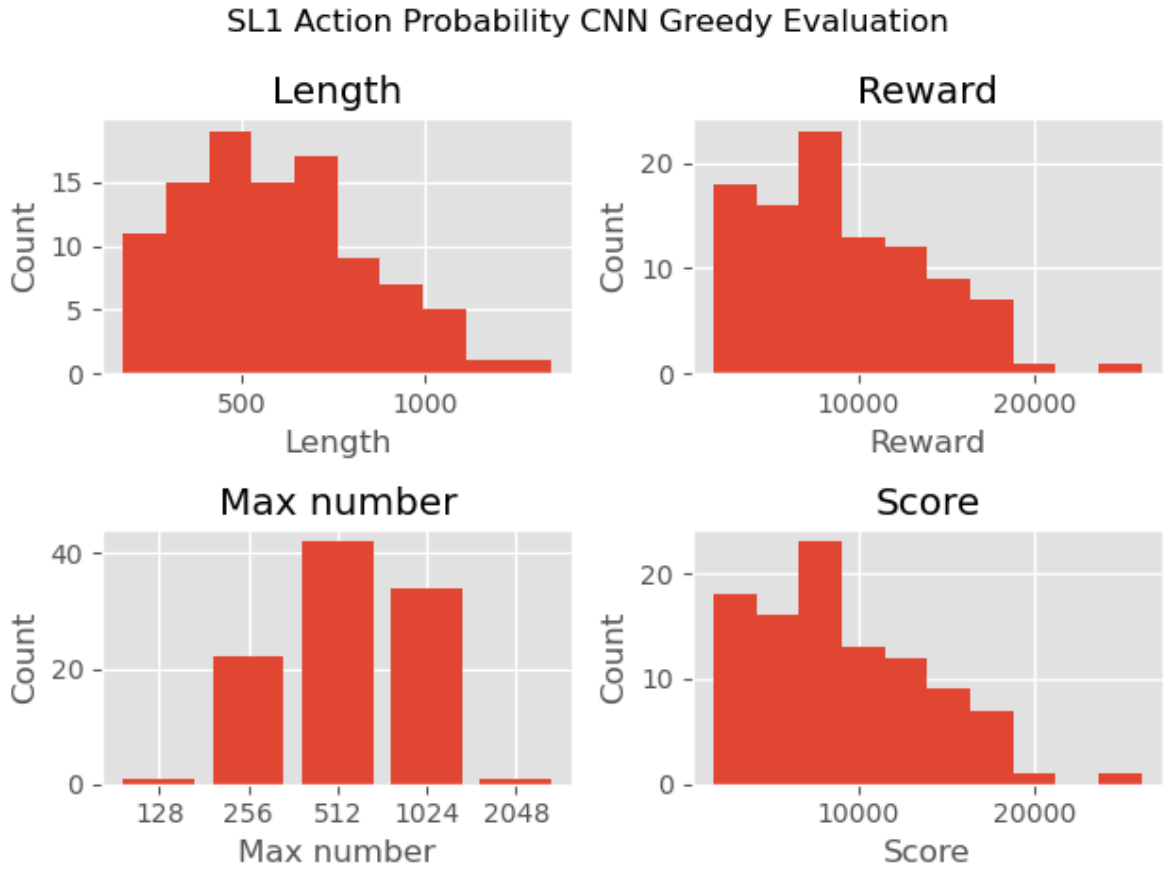


图6-3 SL1 动作概率网络直接贪心策略的评估分布

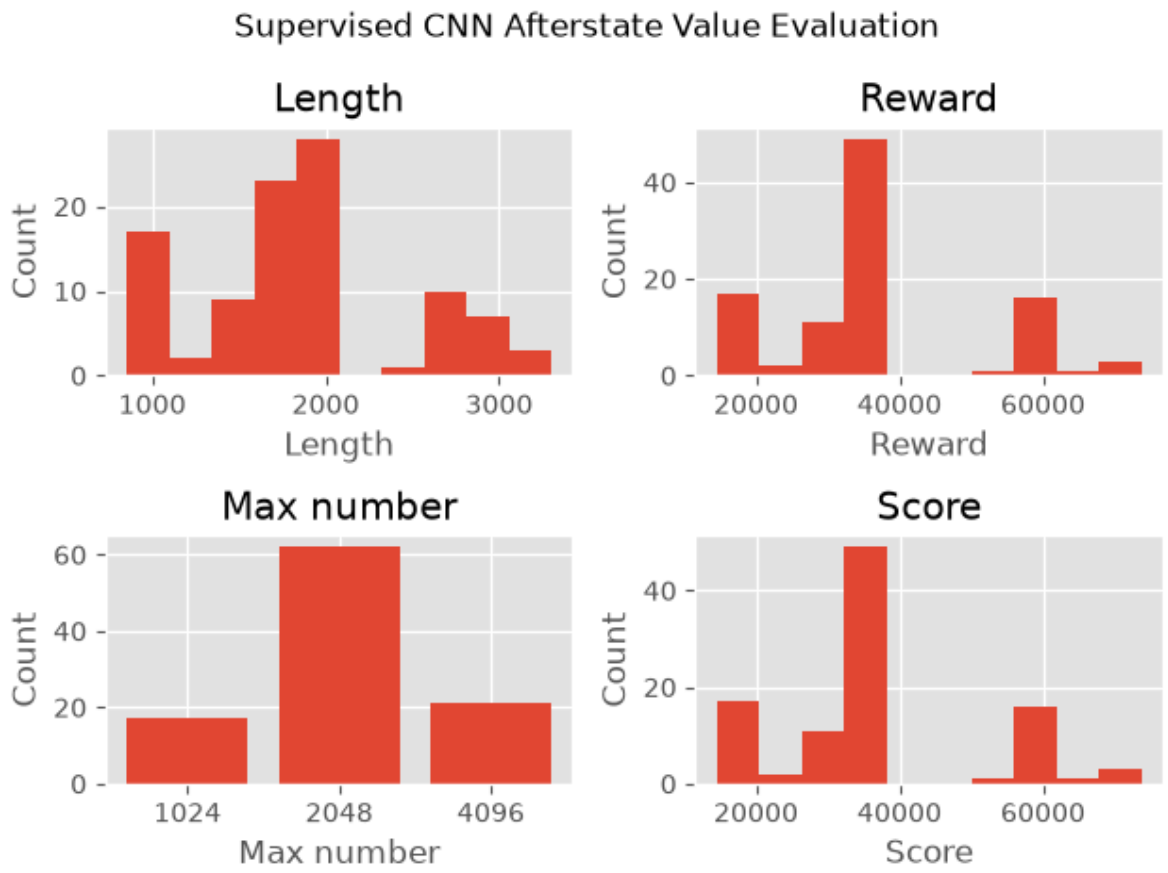


图6-4 SL2 afterstate value 模型结合 depth1 搜索的评估分布

补跑后可以看到，SL1 的真实性能比此前误写的"SL1 + depth1"低得多，均分 `9173.80` 不仅低于 Expectimax depth1，也低于一步启发式贪心。100 局中最高块主要集中在 512 和 1024，只有 1 局达到 2048，说明动作概率网络虽然能从教师数据中学到一些局部动作偏好，但一旦脱离搜索，误差会沿着整局游戏持续累积。旧版报告中"SL1 动作概率模仿 + depth1"的数据来源实际混入了当前分支价值网络相关的 `supervised_cnn_value_eval_depth1.png`，因此会造成"明明只做过 SL1 贪心，却出现 SL1 + depth1 评估"的矛盾；本版已将其更正为 SL1 直接贪心评估。

SL2 的表现明显更好。它只使用 depth1 搜索，却达到了 36659.76 的平均分，超过了本地记录中的 Expectimax depth2 均分 34527.52。更重要的是，SL2 的平均步数达到 1838.55，2048 达成率 83.0%，4096 达成率 21.0%，说明它不是靠少数偶然高分拉高均值，而是整体更稳定。

从理论上看，SL2 成功的原因在于它拟合的是搜索增强后的 $U(x)$，不是动作标签。动作标签是离散的，容易丢失不同合法动作之间的价值差；而 $U(x)$ 是连续标量，能表达"这个 afterstate 比另一个好多少"。这使 CNN 可以成为一个更平滑的叶节点估值器，再由 Expectimax 负责显式处理随机生成节点。

7. 搜索增强监督学习-强化学习框架 RLSL

7.1 思路分析

RLSL 的完整名称可以理解为 Search-Improved Afterstate Value Learning。它借鉴 **AlphaGo Zero** 中"搜索改进策略，再训练网络"的思想，但将 MCTS 换成了更适合 2048 随机环境的 Expectimax。

第 t 轮有当前网络 U_{θ_t}。训练时先用该网络参与搜索，对一批自我对局中的 afterstate 生成搜索增强标签：

$$\hat{U}_t(x) = \mathbb{E}_{s' \sim P(\cdot|x)} \left[\max_a \left(R(s', a) + U_{\theta_t}(T(s', a)) \right) \right]$$

然后用新数据训练网络：

$$\theta_{t+1} = \arg\min_{\theta} \sum_{(x, \hat{U}_t(x))} \ell(U_\theta(x), \hat{U}_t(x))$$

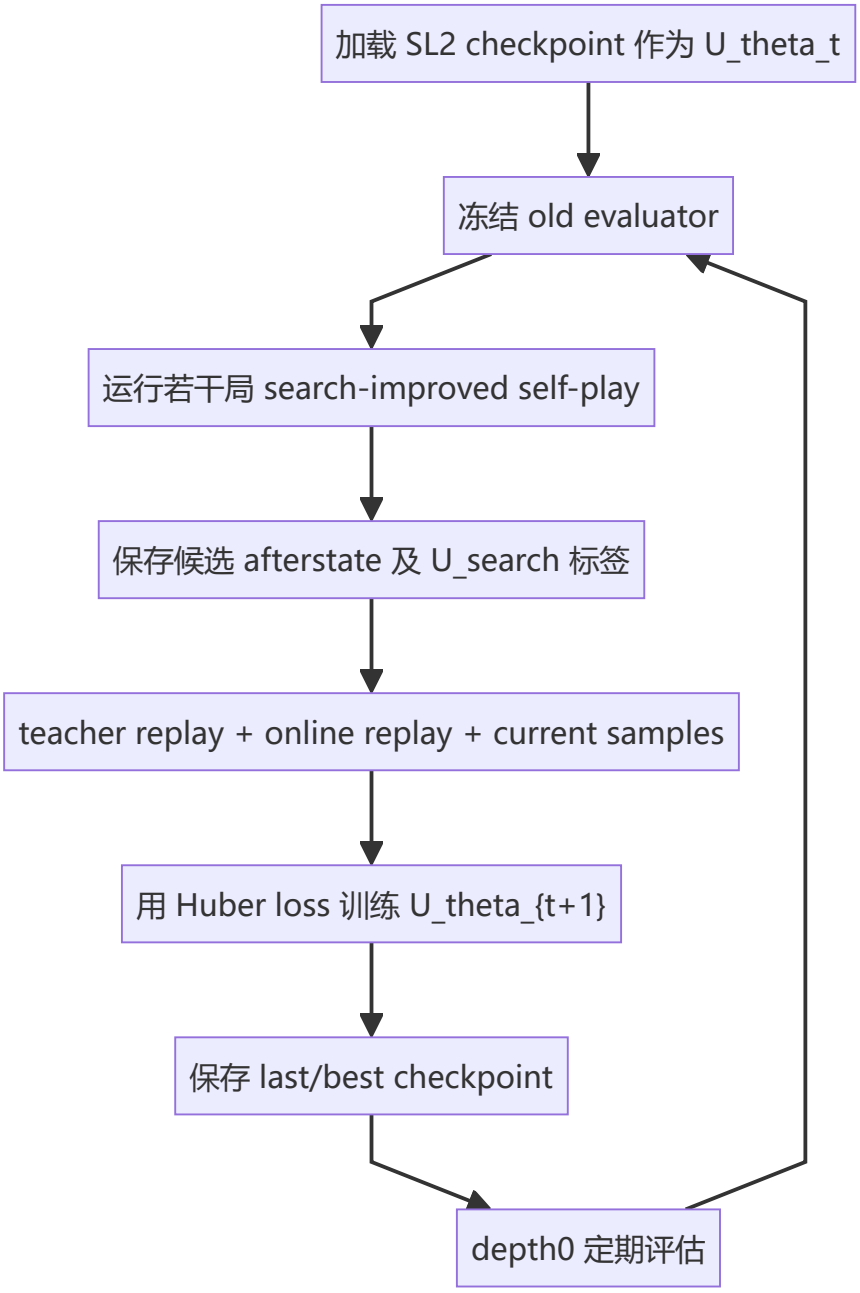
如果一切理想，搜索增强标签应该比原网络更接近 Bellman 最优目标，因此反复迭代后有机会得到更强的价值函数。与普通监督学习相比，RLSL 不再只依赖固定教师，而是让网络通过自我对局不断生成新数据。

7.2 算法架构

RLSL 相关代码在 `gymnasium-2048/src/gymnasium_2048/agents/RLSL/`。当前配置中使用：

配置项	数值
`rounds`	20
`search_depth`	1
`num_eps_per_search`	10
`train_epochs_per_episode`	2
`batch_size`	8900
`learning_rate`	0.0001
`loss_type`	huber
`replay_capacity`	800000
`eval_interval`	1
`eval_episodes`	50

训练流程如下：



一个关键工程细节是 replay buffer。初版 RLSL 只用当前局训练，波动非常大；后续实现加入教师数据锚定和在线 replay，使训练集包含：

$$D_{\text{train}} = D_{\text{teacher}} \cup D_{\text{online}} \cup D_{\text{current}}$$

这样做的目的，是避免当前几局自举数据完全覆盖原本可靠的监督学习分布。

7.3 实验结果与分析

RLSL `multi_eps` 的评估趋势如下。图中绿色虚线是 SL2 depth1 基线均分 36659.76，橙色虚线是 Expectimax depth2 基线均分 34527.52。

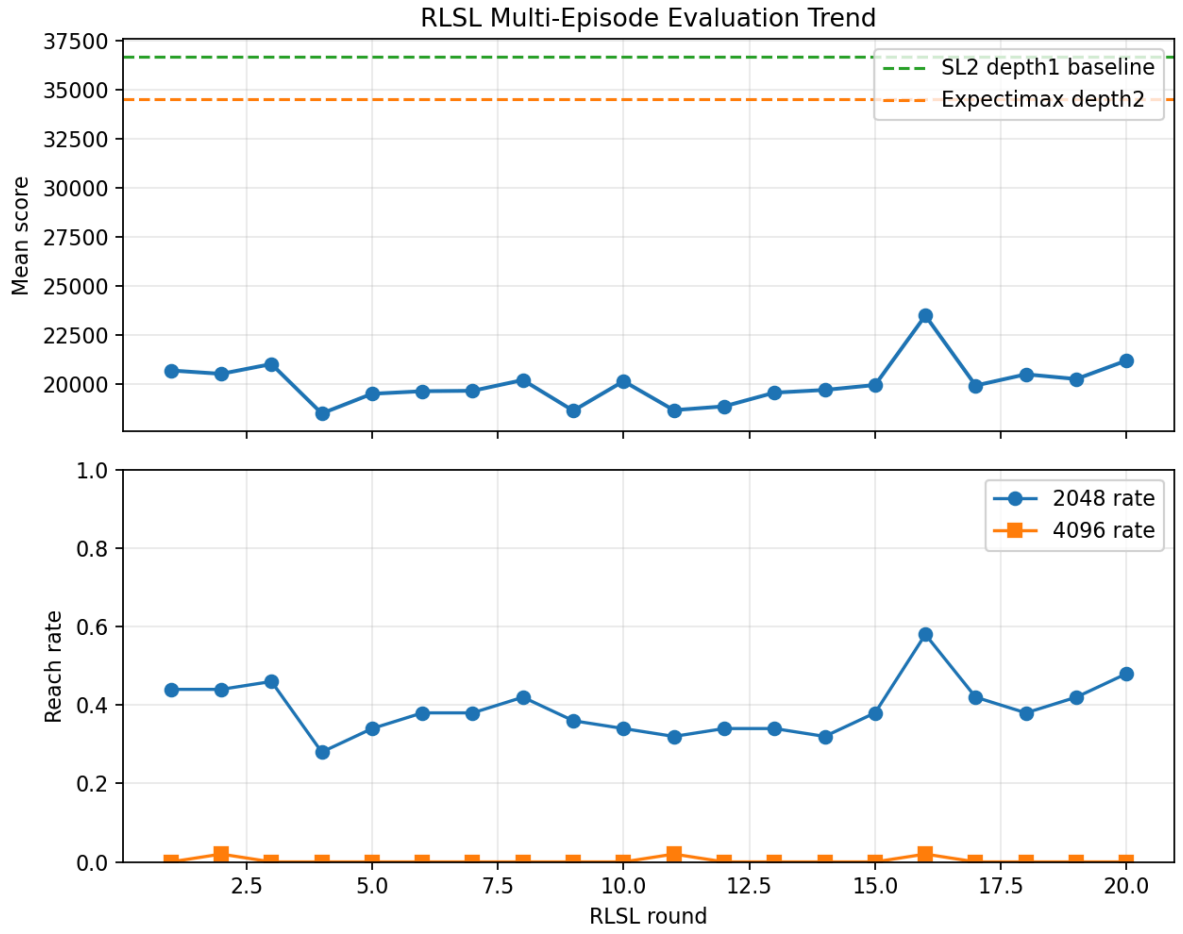


图8-1 RLSL multi_eps 多轮训练过程中的评估趋势

最好的一轮是第 16 轮，50 局 depth0 评估均分为 23497.92，最高分 50928，2048 达成率 58.0%，4096 达成率 2.0%。

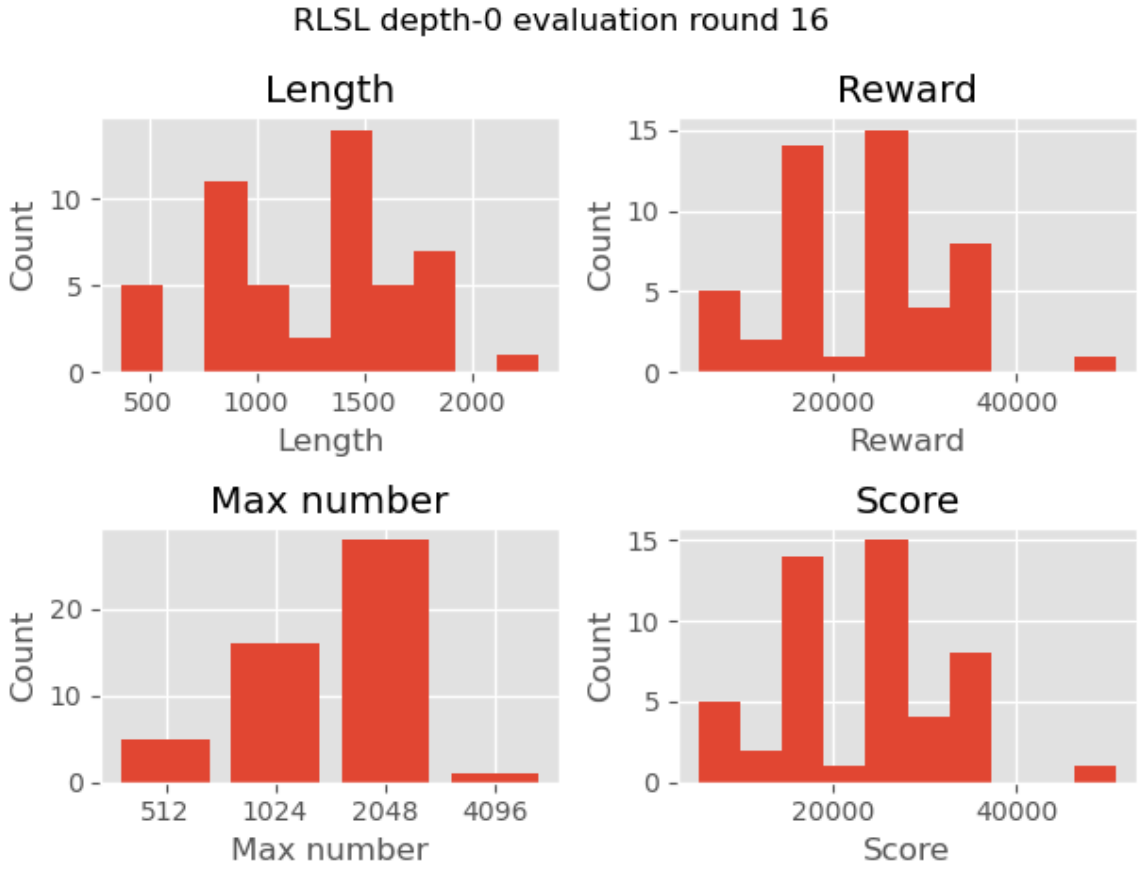


图8-2 RLSL multi_eps 第 16 轮 depth0 评估分布

可以看到，RLSL 没有超过 SL2 初始化模型，也没有达到 Expectimax depth2 的水平。它的曲线在 18k 到 23k 之间波动，没有形成稳定上升趋势。这个现象并不说明思路一定错误，而是说明当前实现还没有处理好自举学习中的稳定性问题。

主要原因可以从强化学习中的 deadly triad 理解：

1. **函数近似**：CNN 只能近似 $U(x)$，如果某些局面分布覆盖不足，误差会在搜索中被放大。
2. **自举**：搜索标签本身使用当前网络 U_{θ_t} 作为叶节点估值器。若 U_{θ_t} 偏差较大，$\hat{U}_t(x)$ 也会带偏。
3. **离策略训练**：replay 中包含教师数据、旧网络数据和当前搜索数据，它们的状态分布并不完全来自最终评估策略。训练分布与测试分布存在偏移。

此外，本实验的 RLSL 还有几个具体问题：

1. 评估使用 depth0，而训练时动作选择使用 search-improved depth1，训练目标和评估方式之间存在落差。
2. 当前网络从很强的 SL2 checkpoint 初始化，随机自举数据不一定比教师数据更优，容易“把好模型训坏”。
3. 2048 后期高价值状态很稀疏，少量 self-play 对局很难覆盖足够多的关键局面。
4. replay capacity 和 current sample admission 需要更细致的比例控制，否则 online 数据既可能太弱，也可能太少。

7.4 后续改进方向

后续可以从以下方向改进：

1. 加入 acceptance gate：只有新模型在固定评估集上超过旧模型，才替换主网络。
2. 使用 target network 或 EMA 慢更新，降低自举目标抖动。
3. 提高 self-play 数据质量：只接纳搜索后价值明显高于旧估计的样本，或按 TD error/局面阶段分桶采样。
4. 统一训练和评估设置：同时报告 depth0 网络裸评估和 depth1 搜索评估。
5. 对后期局面做重采样，增加 2048 之后、4096 之前的高价值状态覆盖。
6. 增大教师数据锚定比例，防止在线自举过早破坏 SL2 已学到的价值结构。

8. 综合讨论

8.1 启发式、搜索、学习之间的关系

本实验中几个算法不是彼此割裂的，而是逐步递进的：

1. 启发式函数提供可解释的初始价值估计；
2. Expectimax 用 Bellman 形式的递推把启发式估值向前展开；
3. 进化算法自动寻找更合适的启发式权重；
4. SL2 用 CNN 拟合 Expectimax 产生的 afterstate 价值，降低深搜索的在线成本；
5. RLSL 试图让 CNN 继续通过自我搜索产生更强监督信号。

从结果上看，最有效的组合是“深搜索教师 + afterstate value 监督学习 + 浅层搜索推理”。这说明 2048 中的价值函数确实存在可学习结构，CNN 可以从大量搜索标签中学习到比人工启发式更细的局面判断。

8.2 效果与成本权衡

方法	在线推理成本	效果	适用场景
一步启发式	低	中等偏低	快速基线、可解释演示
Expectimax depth1	中	较好	常规评估、数据生成
Expectimax depth2	高	很好	教师数据生成、离线评估
SL2 + depth1	中	最好	当前综合最优方案
RLSL	训练成本高	当前不稳定	后续研究方向
N-Tuple TD	训练局数高，推理低	尚未本地完整验证	强化学习对照基线

9. 结论

本实验完成了从启发式搜索到学习型价值函数的完整探索。一步启发式算法提供了可解释的基本策略，但由于只看一步，平均分和 2048 达成率受限。Expectimax 将随机生成节点纳入期望计算，显著提升了策略能力，depth2 相比 depth1 进一步提高了平均分和高阶方块达成率，但计算代价较高。

进化算法证明了评价函数权重可以通过自动搜索得到改进，尤其是空格数、单调性、蛇形结构和即时奖励之间需要平衡。监督学习部分的关键结论是：直接模仿动作概率的 SL1 在裸贪心评估下效果有限，而拟合 afterstate value 的 SL2 更符合 Bellman 最优方程，能把 depth2 教师的知识压缩进 CNN，使 depth1 搜索达到非常强的表现。RLSL 框架在理论上有潜力，但当前实现受函数近似、自举和离策略训练共同影响，没有稳定超过 SL2 初始化模型。

如果继续完善，本项目最值得投入的方向是围绕 SL2/RLSL 做稳定化：引入模型替换门控、target network、分布更均衡的 replay、以及更严格的评估协议。这样既能保留搜索增强学习的理论优势，也能减少自举训练把强监督模型"训退化"的风险。

附录 A：复现实验与生成报告资产的命令

以下命令默认在 `E:\AIhomework` 或 `E:\AIhomework\gymnasium-2048` 下运行，Python 环境使用项目指定的 Conda 环境 `learning`。

A.1 评估基准策略

```
python -m scripts.evaluate --agent heuristic
python -m scripts.evaluate --agent expectimax
python -m scripts.evaluate --agent supervised_cnn
```

A.2 训练或复现进化算法

```
# 一步启发式权重进化
python -m gymnasium_2048.agents.evolution.run_evolution `
  --agent heuristic `
  --generations 13 `
  --population-size 20 `
  --episodes 50 `
  --workers 8 `
  --seed 42 `
  --out-dir models/evolution/heuristic

# Expectimax 分阶段进化
python -m gymnasium_2048.agents.evolution.run_expectimax_staged `
```

```
  --seed 42 `
  --workers 8 `
  --out-dir models/evolution/expectimax_staged
```

A.3 生成 Expectimax 教师数据

```
python -m scripts.generate_expectimax_data `
  --episodes 400 `
  --depth 2 `
  --workers 200 `
  --out data/expectimax_afterstates_400.npz `
  --seed 42
```

这一步计算量较大，本报告使用的是已有数据集 `gymnasium-2048/data/expectimax_afterstates_400.npz`。

A.4 训练 SL2 afterstate value CNN

```
python -m scripts.train --agent supervised_cnn
```

正式长训练配置保存在 `models/supervised_cnn/cnn_400eps_long/training_config.json`。

A.5 训练 RLSL

```
python -m scripts.train --agent RLSL
```

当前 RLSL 配置见 `src/gymnasium_2048/agents/RLSL/configs/train.yaml`。

附录 B：报告资产清单

本报告中的图片引用均来自 `report/assets/`：

文件	来源
`expectimax_depth0.png`	`gymnasium-2048/figures/Expectimax_eval_depth0_hand_defaultweights.png`
`expectimax_depth1.png`	`gymnasium-2048/figures/Expectimax_eval_depth1_hand.png`
`expectimax_depth2.png`	`gymnasium-2048/figures/Expectimax_eval_depth2_hand_defaultweights.png`
`heuristic_evolution_weights.png`	`gymnasium-2048/models/evolution_large/parameter_evolution.png`
`expectimax_evolution_stage1.png`	`gymnasium-2048/models/evolution/expectimax_staged/stage1/parameter_evolution.png`
`expectimax_evolution_stage2.png`	`gymnasium-2048/models/evolution/expectimax_staged/stage2/parameter_evolution.png`
`sl2_cnn_architecture.png`	使用 imagegen 生成的 SL2 CNN 网络结构可视化图
`sl1_greedy_eval.png`	`expectimax` 分支 SL1 动作概率网络补跑评估生成
`sl2_depth1_eval.png`	`gymnasium-2048/figures/supervised_cnn_value_eval_depth1_eps400.png`
`sl2_training_loss.png`	由 `report/scripts/generate_report_assets.py` 生成
`rlsl_multi_eps_curve.png`	由 `report/scripts/generate_report_assets.py` 生成
`rlsl_round16_eval.png`	`gymnasium-2048/models/RLSL/multi_eps/evaluate_16/evaluation.png`